

Resolução do Problema *Counting Rooms* com Teoria dos Grafos

Modelagem, Representação Computacional e Identificação de Componentes Conexos via DFS

Alunos: Samuel Ribeiro Benicio – 2310281

Adriel Medeiros Lins – 2013246

Manoel Sergio Costa Lima – 2422973

Orientador: Prof. Me. Ricardo Carubbi

Disciplina: Métodos Quantitativos

Centro de Ciências Tecnológicas – CCT

Universidade de Fortaleza – UNIFOR



O problema Counting Rooms

Enunciado, Regras e Restrições

- **Cenário:** Mapa em grade bidimensional de dimensões $n \times m$ composto por paredes (#) intransitáveis e chão (.) transitável.
- **Objetivo:** Contar a quantidade total de cômodos.
- **Regra de Movimento:** Deslocamento nas 4 direções cardeais (cima, baixo, esquerda, direita) exclusivamente sobre células transitáveis (.).
- **Restrições Assintóticas:**
 - $1 \leq n, m \leq 1000 \Rightarrow$ grade de até 10^6 células.





Modelagem em Grafos

Mapeamento do Mapa em Grafo Formal

- **Vértices:** cada célula (i, j) da grade vira um vértice, com id único $i \cdot m + j$ (0 a $n \cdot m - 1$).
- **Arestas:** se a célula é chão (.), criamos aresta não-direcionada com o vizinho da direita e o de baixo quando também forem chão — os sentidos esquerda/cima já ficam cobertos quando esse vizinho for processado.
- **Paredes (#):** não geram vértice conectado a nada — ficam isoladas no grafo, fora de qualquer busca.
- **Representação:** lista de adjacência (classes Graph e IntBag), evitando uma matriz $n \cdot m \times n \cdot m$, inviável em memória para grades grandes.
- **Redução do problema:** contar cômodos = contar componentes conexas do grafo resultante.





Representação estrutural

Escolha da Estrutura de Dados e Biblioteca

- **Análise Comparativa:**

- *Matriz de Adjacência*: Inviável ($O(V^2) = 10^{12}$ posições de memória, estouro de RAM).
- *Lista de Arestas*: Ineficiente para busca de vizinhos ($O(E)$ para iterar).
- *Lista de Adjacência*: Complexidade ótima de espaço linear $O(V + E)$ e consulta a vizinhos em $O(\text{grau}(v))$.

- **Objetivo:** Contar a quantidade total de cômodos.

- **Regra de Movimento:** Deslocamento nas 4 direções cardeais (cima, baixo, esquerda, direita) exclusivamente sobre células transitáveis (.).

- **Restrições Assintóticas:**

- $1 \leq n, m \leq 1000 \Rightarrow$ grade de até 10^6 células.





Algoritmo e Complexidade

Busca em Profundidade (DFS) para Contar Componentes

- **Estratégia:** para cada célula de chão ainda não visitada, incrementamos o contador de cômodos e disparamos uma DFS a partir dela, marcando toda a componente conexa como visitada.
- **DFS iterativa:** pilha explícita (array de int) em vez de recursão — uma única sala pode ter até 10^6 células, e a versão recursiva estourava a pilha de chamadas (StackOverflowError) nesse tamanho.
- **Complexidade de tempo:** $O(n \cdot m)$, pois cada vértice e cada aresta são visitados uma única vez.
- **Complexidade de espaço:** $O(n \cdot m)$, para o vetor de visitados, a pilha da DFS e a lista de adjacência.
- **Otimização:** adjacências guardadas em int primitivo (IntBag), evitando o autoboxing de `Bag<Integer>` — necessário para caber no limite de 1s com até 2×10^6 arestas.





Resultados e Conclusão

Validação da Solução e Aprendizados

- **Validação:** solução testada com o exemplo do enunciado (grade 5×8, resultado 3) e aceita no CSES, dentro do limite de tempo (1.00s) e memória (512 MB).
- **Corretude:** a contagem de componentes conexas via DFS corresponde exatamente à contagem de cômodos do mapa.
- **Aprendizado principal:** a escolha da representação (grafo implícito vs. explícito) e da estratégia de busca (iterativa vs. recursiva) tem impacto direto na viabilidade da solução em grades grandes.
- **Aplicabilidade:** o mesmo modelo (grade → grafo → componentes conexas) se generaliza para outros problemas de flood fill, labirintos e mapas.

